SphereSfM to Perspective Images: Camera Pose Transfer Implementation Specification

Document Version: 1.0
Target: AI Agent Coding Implementation
Date: February 3, 2026
Author: Technical Specification for 360° Photogrammetry Pipeline

---

Executive Summary

This document provides complete implementation specifications for transferring camera poses from SphereSfM-aligned equirectangular images to perspective image crops. The goal is to reuse Structure-from-Motion (SfM) results (camera poses, 3D points, features) from equirectangular images and apply them to perspective projections extracted from those images, eliminating redundant SfM computation.

Core principle: Camera pose (world position and orientation) is projection-independent. A perspective crop from an equirectangular image inherits that image's world-space pose, differing only in intrinsic parameters (focal length, field of view).

---

1. System Architecture

1.1 Pipeline Overview

1. **Input:** SphereSfM reconstruction results (sparse/0 folder, database.db)
2. **Process:**
    - Extract perspective images from equirectangular using defined crop parameters
    - Compute perspective camera intrinsics from crop parameters
    - Transfer camera poses from equirectangular to perspective images
    - Create new COLMAP database/reconstruction with perspective images
3. **Output:** COLMAP reconstruction with perspective images + inherited poses

1.2 Data Flow Diagram

```
[SphereSfM Results]
├─ sparse/0/
│  ├─ cameras.bin (SPHERE camera model)
│  ├─ images.bin (ERP image poses)
│  └─ points3D.bin (3D structure)
└─ database.db (features, matches)
↓
[Perspective Extraction Module]
```

```
├─ Extract N crops per ERP image
├─ Compute perspective intrinsics (f, cx, cy)
└─ Generate perspective images
↓
[Pose Transfer Module]
├─ Read ERP image pose (quat + translation)
├─ Assign pose to all perspective crops from same ERP
└─ Create perspective camera models (PINHOLE)
↓
[COLMAP Database Creation]
├─ Insert cameras (PINHOLE model)
├─ Insert images with inherited poses
└─ Optional: Insert features/matches
↓
[Output: Perspective Reconstruction]
├─ sparse_perspective/0/
│  ├─ cameras.bin (PINHOLE models)
│  ├─ images.bin (perspective poses)
│  └─ points3D.bin (optional, inherited)
└─ database_perspective.db
```

---

2. Mathematical Foundation

2.1 Equirectangular Projection

Definition: Maps 3D sphere to 2D plane using latitude-longitude coordinates.

Spherical to ERP pixel mapping:

$$u_{\text{erp}} = \frac{\phi}{2\pi} \times W_{\text{erp}}$$

$$v_{\text{erp}} = \frac{\theta}{\pi} \times H_{\text{erp}}$$

Where:

- $\phi \in [0, 2\pi]$: azimuth angle (longitude)
- $\theta \in [0, \pi]$: polar angle (latitude)
- $(W_{\text{erp}}, H_{\text{erp}})$: equirectangular image dimensions

2.2 Perspective Projection

Pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X_c/Z_c \\ Y_c/Z_c \\ 1 \end{bmatrix}$$

Intrinsic matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

For square pixels: $f_x = f_y = f$

SIMPLE_PINHOLE parameters: $[f, c_x, c_y]$

2.3 Equirectangular to Perspective Transformation

Given crop parameters:

- FOV_h: horizontal field of view (degrees)
- θ_0: pan angle (yaw, rotation around vertical axis)
- ϕ_0: tilt angle (pitch, rotation around horizontal axis)
- (W_p, H_p): output perspective image dimensions

Step 1: Compute focal length

$$f = \frac{W_p/2}{\tan(\text{FOV}_h/2)}$$

For 90° FOV and square image: $f = W_p/2$

Step 2: For each output pixel (u, v) in perspective image:

1. Convert to normalized image coordinates:
$$x = \frac{u - c_x}{f}, \quad y = \frac{v - c_y}{f}$$
2. Create 3D ray in camera space:
$$\mathbf{r}_{\text{cam}} = [x, y, 1]^T \quad \text{(normalized)}$$
3. Convert to spherical coordinates:
$$\theta = \arctan 2(\sqrt{x^2 + y^2}, 1)$$
$$\phi = \arctan 2(y, x)$$
4. Apply viewing direction offset:
$$\theta' = \theta + \theta_0, \quad \phi' = \phi + \phi_0$$
5. Map to equirectangular pixel:
$$u_{\text{erp}} = \frac{\phi'}{2\pi} \times W_{\text{erp}}$$
$$v_{\text{erp}} = \frac{\theta'}{\pi} \times H_{\text{erp}}$$
6. Sample color from equirectangular image at $(u_{\text{erp}}, v_{\text{erp}})$

2.4 Camera Pose Representation

COLMAP uses world-to-camera transformation:

$$\mathbf{T}_{\text{cam}}^{\text{world}} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}$$

Where:

- $\mathbf{R} \in SO(3)$: rotation matrix (3×3)
- $\mathbf{t} \in \mathbb{R}^3$: translation vector

Quaternion representation: $\mathbf{q} = [w, x, y, z]$

Conversion quaternion → rotation matrix:

$$\mathbf{R} = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

Camera center in world coordinates:

$$\mathbf{C}_{\text{world}} = -\mathbf{R}^T \mathbf{t}$$

---

3. Implementation Strategy

3.1 Strategy Comparison

Strategy	Speed	Accuracy	Complexity	Use Case
Direct Inheritance	⭑⭑⭑⭑⭑	⭑⭑⭑	Low	Quick prototyping
Bundle Adjustment	⭑⭑⭑	⭑⭑⭑⭑⭑	Medium	Production
Feature Reuse	⭑⭑⭑⭑	⭑⭑⭑⭑	High	Validation

Table 1: Implementation strategy comparison

3.2 Recommended Strategy: Direct Inheritance with Optional Refinement

Rationale:

- Fastest initial results
- Mathematically correct (pose is view-independent)
- Allows optional refinement via bundle adjustment
- Minimal implementation complexity

---

4. Module Specifications

4.1 Module 1: Perspective Image Extractor

Filename: `perspective_extractor.py`

Responsibilities:

- Load equirectangular images
- Extract perspective crops using specified parameters
- Save perspective images to disk

Input Parameters:

- `equirectangular_image_path`: Path to ERP image
- `crop_configs`: List of dictionaries with keys:
  - `fov`: Horizontal field of view (degrees)
  - `theta`: Pan angle (degrees, 0=forward, +90=right, -90=left)
  - `phi`: Tilt angle (degrees, 0=horizontal, +45=up, -45=down)
  - `width`: Output image width (pixels)
  - `height`: Output image height (pixels)
- `output_directory`: Where to save perspective images

Output:

- Perspective images saved as JPG/PNG
- Returns list of output image paths

Implementation using Equirec2Perspec library:

```
import cv2
import numpy as np
from Equirec2Perspec import Equirectangular

class PerspectiveExtractor:
def init(self, equirectangular_image_path):
"""
Initialize with path to equirectangular image.

    Args:
        equirectangular_image_path: Path to ERP image file
    """
    self.equ = Equirectangular(equirectangular_image_path)
    self.image_name = os.path.basename(equirectangular_image_path)

def extract_crops(self, crop_configs, output_directory):
    """
    Extract perspective crops from equirectangular image.
```

```
    Args:
        crop_configs: List of dicts with keys:
            - fov: Horizontal FOV in degrees
            - theta: Pan angle in degrees
            - phi: Tilt angle in degrees
            - width: Output width in pixels
            - height: Output height in pixels
        output_directory: Directory to save perspective images

    Returns:
        List of tuples: (output_path, crop_config, intrinsics)
    """
    os.makedirs(output_directory, exist_ok=True)
    results = []

    for i, config in enumerate(crop_configs):
        # Extract perspective image
        persp_img = self.equ.GetPerspective(
            FOV=config['fov'],
            Theta=config['theta'],
            Phi=config['phi'],
            Hd=config['height'],
            Wd=config['width']
        )

        # Generate output filename
        base_name = os.path.splitext(self.image_name)[0]
        output_name = f"{base_name}_persp_{i:03d}.jpg"
        output_path = os.path.join(output_directory, output_name)

        # Save image
        cv2.imwrite(output_path, persp_img)

        # Compute intrinsics
        intrinsics = self.compute_intrinsics(
            config['fov'],
            config['width'],
```

```
            config['height']
        )

        results.append((output_path, config, intrinsics))

    return results

@staticmethod
def compute_intrinsics(fov_degrees, width, height):
    """
    Compute perspective camera intrinsics from FOV.

    Args:
        fov_degrees: Horizontal field of view in degrees
        width: Image width in pixels
        height: Image height in pixels

    Returns:
        Dictionary with keys: fx, fy, cx, cy
    """
    fov_rad = np.deg2rad(fov_degrees)
    focal_length = (width / 2.0) / np.tan(fov_rad / 2.0)

    return {
        'fx': focal_length,
        'fy': focal_length,  # Assume square pixels
        'cx': width / 2.0,
        'cy': height / 2.0,
        'width': width,
        'height': height
    }
```

Example usage

if **name** == "**main**":
# Define crop configurations
crops = [
{'fov': 90, 'theta': 0, 'phi': 0, 'width': 1024, 'height': 1024}, # Forward
{'fov': 90, 'theta': 90, 'phi': 0, 'width': 1024, 'height': 1024}, # Right

{'fov': 90, 'theta': 180, 'phi': 0, 'width': 1024, 'height': 1024}, # Back
{'fov': 90, 'theta': -90, 'phi': 0, 'width': 1024, 'height': 1024}, # Left
{'fov': 90, 'theta': 0, 'phi': 45, 'width': 1024, 'height': 1024}, # Up
{'fov': 90, 'theta': 0, 'phi': -45, 'width': 1024, 'height': 1024}, # Down
]

```
extractor = PerspectiveExtractor('path/to/equirectangular.jpg')
results = extractor.extract_crops(crops, 'output/perspective_images')

print(f"Extracted {len(results)} perspective images")
```

Alternative: Custom Implementation (No External Library)

import cv2
import numpy as np

class CustomPerspectiveExtractor:
def **init**(self, equirectangular_image):
"""
Args:
equirectangular_image: NumPy array (H, W, 3) in BGR format
"""
self.erp_image = equirectangular_image
self.erp_height, self.erp_width = equirectangular_image.shape[:2]

```
def extract_perspective(self, fov_deg, theta_deg, phi_deg, width, height):
    """
    Extract perspective view from equirectangular image.

    Args:
        fov_deg: Horizontal field of view (degrees)
        theta_deg: Pan angle (degrees)
        phi_deg: Tilt angle (degrees)
        width: Output image width
        height: Output image height

    Returns:
        NumPy array (height, width, 3) perspective image
    """
    # Convert angles to radians
    fov = np.deg2rad(fov_deg)
    theta = np.deg2rad(theta_deg)
```

```
phi = np.deg2rad(phi_deg)

# Compute focal length
f = (width / 2.0) / np.tan(fov / 2.0)
cx, cy = width / 2.0, height / 2.0

# Create output image
persp_img = np.zeros((height, width, 3), dtype=np.uint8)

# For each pixel in perspective image
for v in range(height):
    for u in range(width):
        # Normalized image coordinates
        x = (u - cx) / f
        y = (v - cy) / f

        # 3D ray direction in camera space
        norm = np.sqrt(x**2 + y**2 + 1)
        ray_x, ray_y, ray_z = x / norm, y / norm, 1.0 / norm

        # Apply viewing direction (theta=yaw, phi=pitch)
        # Rotate around Y axis (pitch)
        cos_phi, sin_phi = np.cos(phi), np.sin(phi)
        temp_x = ray_x * cos_phi + ray_z * sin_phi
        temp_z = -ray_x * sin_phi + ray_z * cos_phi

        # Rotate around Z axis (yaw)
        cos_theta, sin_theta = np.cos(theta), np.sin(theta)
        final_x = temp_x * cos_theta - ray_y * sin_theta
        final_y = temp_x * sin_theta + ray_y * cos_theta
        final_z = temp_z

        # Convert to spherical coordinates
        lon = np.arctan2(final_y, final_x)  # [-pi, pi]
        lat = np.arctan2(final_z, np.sqrt(final_x**2 + final_y**2))  # [-pi/2, pi/2]

        # Map to equirectangular pixel coordinates
        u_erp = int((lon + np.pi) / (2 * np.pi) * self.erp_width) % self.erp_width
```

```
            v_erp = int((np.pi/2 - lat) / np.pi * self.erp_height)
            v_erp = np.clip(v_erp, 0, self.erp_height - 1)

            # Sample pixel
            persp_img[v, u] = self.erp_image[v_erp, u_erp]

    return persp_img
```

4.2 Module 2: SphereSfM Pose Reader

Filename: `spheresfm_reader.py`

Responsibilities:

- Read COLMAP reconstruction from SphereSfM output
- Extract camera poses (quaternion + translation)
- Parse camera intrinsics
- Map image names to poses

Input:

- Path to SphereSfM sparse reconstruction folder (`sparse/0`)

Output:

- Dictionary mapping image names to pose data

Implementation using pycolmap:

```
import pycolmap
import numpy as np
from dataclasses import dataclass
from typing import Dict, Tuple

@dataclass
class CameraPose:
"""Camera pose representation."""
quaternion: np.ndarray # [w, x, y, z]
translation: np.ndarray # [tx, ty, tz]
camera_id: int
image_id: int
```

```
    def to_rotation_matrix(self):
        """Convert quaternion to rotation matrix."""
        w, x, y, z = self.quaternion
        R = np.array([
            [1-2*(y**2+z**2), 2*(x*y-w*z), 2*(x*z+w*y)],
```

```
        [2*(x*y+w*z), 1-2*(x**2+z**2), 2*(y*z-w*x)],
        [2*(x*z-w*y), 2*(y*z+w*x), 1-2*(x**2+y**2)]
    ])
    return R

def camera_center_world(self):
    """Compute camera center in world coordinates."""
    R = self.to_rotation_matrix()
    return -R.T @ self.translation
```

class SphereSfMReader:
def **init**(self, sparse_path):
"""
Initialize reader with SphereSfM reconstruction.

```
    Args:
        sparse_path: Path to sparse/0 folder from SphereSfM
    """
    self.reconstruction = pycolmap.Reconstruction(sparse_path)
    self.poses = self._extract_poses()

def _extract_poses(self) -> Dict[str, CameraPose]:
    """
    Extract all image poses from reconstruction.

    Returns:
        Dictionary mapping image name to CameraPose
    """
    poses = {}

    for image_id, image in self.reconstruction.images.items():
        if not image.has_pose:
            continue

        # Get camera-from-world transformation
        cam_from_world = image.cam_from_world()

        # Extract rotation (as quaternion) and translation
```

```
        quat = cam_from_world.rotation.quat  # [w, x, y, z]
        trans = cam_from_world.translation

        pose = CameraPose(
            quaternion=np.array([quat[0], quat[1], quat[2], quat[3]]),
            translation=np.array([trans[0], trans[1], trans[2]]),
            camera_id=image.camera_id,
            image_id=image_id
        )

        poses[image.name] = pose

    return poses

def get_pose(self, image_name: str) -> CameraPose:
    """
    Get pose for specific image.

    Args:
        image_name: Name of image (e.g., 'IMG_0001.jpg')

    Returns:
        CameraPose object
    """
    if image_name not in self.poses:
        raise ValueError(f"Image {image_name} not found in reconstruction")
    return self.poses[image_name]

def get_camera(self, camera_id: int):
    """Get camera intrinsics for given camera ID."""
    return self.reconstruction.cameras[camera_id]
```

Example usage

if **name** == "**main**":
reader = SphereSfMReader('colmap/sparse/0')

```
# Get pose for specific image
pose = reader.get_pose('IMG_0001.jpg')
print(f"Quaternion: {pose.quaternion}")
print(f"Translation: {pose.translation}")
print(f"Camera center (world): {pose.camera_center_world()}")
```

Alternative: Reading from text files (if pycolmap unavailable)

```
import numpy as np
from dataclasses import dataclass
from typing import Dict

@dataclass
class CameraPose:
quaternion: np.ndarray # [w, x, y, z]
translation: np.ndarray # [tx, ty, tz]
camera_id: int
image_id: int

class TextFileReader:
@staticmethod
def read_images_txt(images_txt_path) -> Dict[str, CameraPose]:
"""
Read images.txt from COLMAP text export.
```

```
    Format:
    # IMAGE_ID, QW, QX, QY, QZ, TX, TY, TZ, CAMERA_ID, NAME
    # POINTS2D[] as (X, Y, POINT3D_ID)

    Args:
        images_txt_path: Path to images.txt

    Returns:
        Dictionary mapping image name to CameraPose
    """
    poses = {}

    with open(images_txt_path, 'r') as f:
        lines = f.readlines()

    i = 0
    while i < len(lines):
```

```
        line = lines[i].strip()

        # Skip comments and empty lines
        if line.startswith('#') or len(line) == 0:
            i += 1
            continue

        # Parse image line
        parts = line.split()
        image_id = int(parts[0])
        qw, qx, qy, qz = map(float, parts[1:5])
        tx, ty, tz = map(float, parts[5:8])
        camera_id = int(parts[8])
        image_name = parts[9]

        pose = CameraPose(
            quaternion=np.array([qw, qx, qy, qz]),
            translation=np.array([tx, ty, tz]),
            camera_id=camera_id,
            image_id=image_id
        )

        poses[image_name] = pose

        # Skip next line (points2D)
        i += 2

    return poses
```

Example usage

poses = TextFileReader.read_images_txt('colmap/sparse/0/images.txt')

4.3 Module 3: Perspective Reconstruction Builder

Filename: `perspective_reconstruction.py`

Responsibilities:

- Create new COLMAP reconstruction for perspective images
- Assign inherited poses to perspective images

- Define PINHOLE camera models
- Write reconstruction to disk

Input:

- List of perspective images with metadata (path, intrinsics, crop config)
- Original equirectangular pose (from SphereSfM)
- Output path for new reconstruction

Output:

- COLMAP reconstruction folder (`sparse_perspective/0`)

Implementation:

```
import pycolmap
import numpy as np
from pathlib import Path
from typing import List, Dict, Tuple

class PerspectiveReconstructionBuilder:
def init(self):
"""Initialize empty reconstruction for perspective images."""
self.reconstruction = pycolmap.Reconstruction()
self.camera_cache = {} # Cache cameras by (width, height, fx)
```

```
def add_camera(self, width, height, fx, fy, cx, cy) -> int:
    """
    Add PINHOLE camera model to reconstruction.

    Args:
        width, height: Image dimensions
        fx, fy: Focal lengths
        cx, cy: Principal point

    Returns:
        camera_id
    """
    # Check cache to reuse cameras with same parameters
    cache_key = (width, height, fx, fy, cx, cy)
    if cache_key in self.camera_cache:
        return self.camera_cache[cache_key]

    # Create PINHOLE camera
    # Model params: [fx, fy, cx, cy]
```

```
    camera = pycolmap.Camera(
        model_name='PINHOLE',
        width=width,
        height=height,
        params=[fx, fy, cx, cy]
    )

    camera_id = self.reconstruction.add_camera(camera)
    self.camera_cache[cache_key] = camera_id

    return camera_id

def add_image_with_pose(self, image_name: str, camera_id: int,
                        pose: 'CameraPose') -> int:
    """
    Add image to reconstruction with inherited pose.

    Args:
        image_name: Name/path of perspective image
        camera_id: ID of associated camera
        pose: Inherited CameraPose from equirectangular

    Returns:
        image_id
    """
    # Create image object
    image = pycolmap.Image(
        name=image_name,
        camera_id=camera_id
    )

    # Create Rigid3d transformation (camera-from-world)
    rotation = pycolmap.Rotation3d(quat=pose.quaternion)
    cam_from_world = pycolmap.Rigid3d(rotation, pose.translation)

    # Set pose
    image.set_cam_from_world(camera_id, cam_from_world)
```

```
    # Add to reconstruction
    image_id = self.reconstruction.add_image(image)

    return image_id

def build_from_perspective_data(self, perspective_data: List[Tuple],
                                equ_pose: 'CameraPose'):
    """
    Build complete perspective reconstruction from extracted data.

    Args:
        perspective_data: List of tuples (image_path, crop_config, intrinsics)
        equ_pose: Original equirectangular image pose to inherit
    """
    for image_path, crop_config, intrinsics in perspective_data:
        # Add camera if not exists
        camera_id = self.add_camera(
            width=intrinsics['width'],
            height=intrinsics['height'],
            fx=intrinsics['fx'],
            fy=intrinsics['fy'],
            cx=intrinsics['cx'],
            cy=intrinsics['cy']
        )

        # Add image with inherited pose
        image_name = Path(image_path).name
        self.add_image_with_pose(image_name, camera_id, equ_pose)

def write(self, output_path: str):
    """
    Write reconstruction to disk.

    Args:
        output_path: Directory to save reconstruction (e.g., 'sparse_perspective/0')
    """
    Path(output_path).mkdir(parents=True, exist_ok=True)
    self.reconstruction.write(output_path)
```

```
    print(f"Reconstruction written to {output_path}")
    print(f"  - {len(self.reconstruction.cameras)} cameras")
    print(f"  - {len(self.reconstruction.images)} images")
```

Example usage

if **name** == "**main**":
from spheresfm_reader import SphereSfMReader
from perspective_extractor import PerspectiveExtractor

```
# Step 1: Read original SphereSfM poses
sfm_reader = SphereSfMReader('colmap/sparse/0')
equ_pose = sfm_reader.get_pose('IMG_0001.jpg')

# Step 2: Extract perspective images
extractor = PerspectiveExtractor('images/IMG_0001.jpg')
crop_configs = [
    {'fov': 90, 'theta': 0, 'phi': 0, 'width': 1024, 'height': 1024},
    # ... more crops
]
perspective_data = extractor.extract_crops(crop_configs, 'perspective_images')

# Step 3: Build perspective reconstruction
builder = PerspectiveReconstructionBuilder()
builder.build_from_perspective_data(perspective_data, equ_pose)
builder.write('sparse_perspective/0')
```

4.4 Module 4: Batch Pipeline Orchestrator

Filename: `pipeline_orchestrator.py`

Responsibilities:

- Process all equirectangular images in batch
- Coordinate extraction, pose transfer, and reconstruction building
- Handle error cases and logging

Implementation:

import os
import json
from pathlib import Path

from typing import List, Dict
import logging

from spheresfm_reader import SphereSfMReader
from perspective_extractor import PerspectiveExtractor
from perspective_reconstruction import PerspectiveReconstructionBuilder

logging.basicConfig(level=logging.INFO,
format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(**name**)

class PipelineOrchestrator:
def **init**(self, config_path: str):
"""
Initialize pipeline with configuration file.

```
    Args:
        config_path: Path to JSON configuration file
    """
    with open(config_path, 'r') as f:
        self.config = json.load(f)

    self.sfm_reader = SphereSfMReader(self.config['spheresfm_sparse_path'])
    self.builder = PerspectiveReconstructionBuilder()

def process_all_images(self):
    """Process all equirectangular images to perspective."""
    equ_image_dir = Path(self.config['equirectangular_image_dir'])
    perspective_output_dir = Path(self.config['perspective_output_dir'])
    perspective_output_dir.mkdir(parents=True, exist_ok=True)

    # Get list of equirectangular images
    equ_images = list(equ_image_dir.glob('*.jpg')) + list(equ_image_dir.glob('*.png'
    logger.info(f"Found {len(equ_images)} equirectangular images")

    crop_configs = self.config['crop_configurations']

    processed_count = 0
    for equ_image_path in equ_images:
        try:
            logger.info(f"Processing {equ_image_path.name}...")
```

```
            # Get pose for this image
            equ_pose = self.sfm_reader.get_pose(equ_image_path.name)

            # Extract perspective crops
            extractor = PerspectiveExtractor(str(equ_image_path))
            perspective_data = extractor.extract_crops(
                crop_configs,
                str(perspective_output_dir)
            )

            # Add to reconstruction
            self.builder.build_from_perspective_data(perspective_data, equ_pose)

            processed_count += 1
            logger.info(f"  -> Extracted {len(perspective_data)} perspective images")

        except Exception as e:
            logger.error(f"Error processing {equ_image_path.name}: {e}")
            continue

    logger.info(f"Processed {processed_count}/{len(equ_images)} images")

    # Write final reconstruction
    output_sparse_path = self.config['output_sparse_path']
    self.builder.write(output_sparse_path)
    logger.info(f"Pipeline complete. Output: {output_sparse_path}")
```

Configuration file example (config.json)

```
CONFIG_TEMPLATE = {
"spheresfm_sparse_path": "colmap/sparse/0",
"equirectangular_image_dir": "images/equirectangular",
"perspective_output_dir": "images/perspective",
"output_sparse_path": "sparse_perspective/0",
"crop_configurations": [
{"fov": 90, "theta": 0, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": 90, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": 180, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": -90, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": 0, "phi": 45, "width": 1024, "height": 1024},
```

{"fov": 90, "theta": 0, "phi": -45, "width": 1024, "height": 1024}
]
}

Example usage

if **name** == "**main**":
# Save config template
with open('config.json', 'w') as f:
json.dump(CONFIG_TEMPLATE, f, indent=2)

```
# Run pipeline
orchestrator = PipelineOrchestrator('config.json')
orchestrator.process_all_images()
```

---

5. COLMAP Database Structure Reference

5.1 Database Tables

cameras table:

images table:

5.2 Camera Model Constants

From COLMAP camera models

CAMERA_MODEL_IDS = {
'SIMPLE_PINHOLE': 0, # params: [f, cx, cy]
'PINHOLE': 1, # params: [fx, fy, cx, cy]
'SIMPLE_RADIAL': 2, # params: [f, cx, cy, k]
'RADIAL': 3, # params: [f, cx, cy, k1, k2]
'OPENCV': 4, # params: [fx, fy, cx, cy, k1, k2, p1, p2]
'SPHERE': 99, # params: [dummy, width, height] (SphereSfM)
}

5.3 Binary Format Structures

images.bin format:

cameras.bin format:

---

6. Error Handling and Edge Cases

6.1 Common Error Scenarios

- **Missing image in SphereSfM**: Skip perspective extraction for that image
- **Invalid crop parameters**: Validate FOV $\in [10^\circ, 180^\circ]$, theta/phi $\in [-180^\circ, 180^\circ]$
- **Memory constraints**: Process images in batches if dataset is large
- **Duplicate camera intrinsics**: Reuse camera IDs to reduce database size

6.2 Validation Checks

def validate_crop_config(config: Dict) -> bool:
"""
Validate crop configuration parameters.

```
    Args:
        config: Dictionary with fov, theta, phi, width, height

    Returns:
        True if valid, False otherwise
    """
    checks = [
        10 <= config['fov'] <= 180,
        -180 <= config['theta'] <= 180,
        -90 <= config['phi'] <= 90,
        config['width'] > 0,
        config['height'] > 0
    ]

    if not all(checks):
        logger.warning(f"Invalid crop config: {config}")
        return False

    return True
```

6.3 Memory Optimization

For large datasets (>1000 images):

def process_in_batches(image_list, batch_size=50):
"""
Process images in batches to manage memory.

```
    Args:
        image_list: List of image paths
        batch_size: Number of images per batch

    Yields:
        Batches of image paths
    """
    for i in range(0, len(image_list), batch_size):
        yield image_list[i:i+batch_size]
```

---

7. Testing and Validation

7.1 Unit Tests

Test perspective extraction:

```
import unittest
import numpy as np
from perspective_extractor import PerspectiveExtractor

class TestPerspectiveExtractor(unittest.TestCase):
def test_intrinsics_computation(self):
"""Test focal length computation from FOV."""
intrinsics = PerspectiveExtractor.compute_intrinsics(
fov_degrees=90,
width=1024,
height=1024
)
```

```
        # For 90° FOV, f = width/2
        expected_f = 1024 / 2.0
        self.assertAlmostEqual(intrinsics['fx'], expected_f, places=2)
        self.assertAlmostEqual(intrinsics['cx'], 512.0, places=2)
        self.assertAlmostEqual(intrinsics['cy'], 512.0, places=2)

    def test_fov_range(self):
        """Test intrinsics for various FOV values."""
        for fov in [60, 90, 120]:
            intrinsics = PerspectiveExtractor.compute_intrinsics(fov, 1024, 1024)
            self.assertGreater(intrinsics['fx'], 0)
```

if **name** == '**main**':
unittest.main()

Test pose inheritance:

class TestPoseInheritance(unittest.TestCase):
def test_pose_transfer(self):
"""Test that perspective images inherit equirectangular pose."""
# Create mock equirectangular pose
equ_pose = CameraPose(
quaternion=np.array([1.0, 0.0, 0.0, 0.0]),
translation=np.array([1.0, 2.0, 3.0]),
camera_id=1,
image_id=1
)

```
# Build perspective reconstruction
builder = PerspectiveReconstructionBuilder()
camera_id = builder.add_camera(1024, 1024, 512, 512, 512, 512)
image_id = builder.add_image_with_pose('test.jpg', camera_id, equ_pose)

# Verify pose was inherited
recon_image = builder.reconstruction.images[image_id]
self.assertTrue(recon_image.has_pose)

# Check translation matches
cam_from_world = recon_image.cam_from_world()
np.testing.assert_array_almost_equal(
    cam_from_world.translation,
    equ_pose.translation
)
```

7.2 Integration Test

End-to-end pipeline test:

def test_complete_pipeline():
"""
Test complete pipeline from SphereSfM to perspective reconstruction.
"""
# Setup test data
test_config = {
"spheresfm_sparse_path": "test_data/sparse/0",
"equirectangular_image_dir": "test_data/equirectangular",
"perspective_output_dir": "test_output/perspective",

"output_sparse_path": "test_output/sparse_perspective/0",
"crop_configurations": [
{"fov": 90, "theta": 0, "phi": 0, "width": 512, "height": 512}
]
}

```
# Save config
with open('test_config.json', 'w') as f:
    json.dump(test_config, f)

# Run pipeline
orchestrator = PipelineOrchestrator('test_config.json')
orchestrator.process_all_images()

# Verify output exists
assert os.path.exists('test_output/sparse_perspective/0/cameras.bin')
assert os.path.exists('test_output/sparse_perspective/0/images.bin')

# Verify reconstruction
recon = pycolmap.Reconstruction('test_output/sparse_perspective/0')
assert len(recon.images) > 0
assert len(recon.cameras) > 0

print("✓ Integration test passed")
```

7.3 Visual Validation

COLMAP GUI inspection:

Visualize perspective reconstruction

colmap gui
--database_path database_perspective.db
--image_path images/perspective
--import_path sparse_perspective/0

Check for issues:

- All images positioned at correct locations
- Camera frustums aligned with expected viewing directions
- No outliers or misaligned cameras

---

8. Optional: Bundle Adjustment Refinement

8.1 When to Apply Bundle Adjustment

Recommended if:

- Equirectangular alignment has small uncertainties
- Perspective crops have significant overlap
- High-precision pose estimates required

Skip if:

- Equirectangular poses are highly accurate
- Time constraints exist
- Perspective crops have minimal overlap

8.2 Bundle Adjustment Command

After creating perspective reconstruction with inherited poses

colmap bundle_adjuster
--input_path sparse_perspective/0
--output_path sparse_perspective/0_refined
--BundleAdjustment.refine_focal_length 0
--BundleAdjustment.refine_principal_point 0
--BundleAdjustment.refine_extra_params 0
--BundleAdjustment.refine_extrinsics 1

Parameters:

- `refine_focal_length=0`: Keep computed focal lengths fixed
- `refine_extrinsics=1`: Allow pose refinement
- Result: Small corrections to inherited poses

8.3 Python Integration

def refine_poses_bundle_adjustment(reconstruction_path, output_path):
"""
Apply bundle adjustment to refine inherited poses.

```
    Args:
        reconstruction_path: Path to initial perspective reconstruction
        output_path: Path to save refined reconstruction
    """
    recon = pycolmap.Reconstruction(reconstruction_path)
```

```
# Configure bundle adjustment
options = pycolmap.BundleAdjustmentOptions()
options.refine_focal_length = False  # Keep intrinsics fixed
options.refine_principal_point = False
options.refine_extra_params = False
options.refine_extrinsics = True  # Refine poses

# Run bundle adjustment
logger.info("Running bundle adjustment...")
pycolmap.bundle_adjust(recon, options)

# Save refined reconstruction
recon.write(output_path)
logger.info(f"Refined reconstruction saved to {output_path}")
```

---

9. Performance Considerations

9.1 Computational Complexity

Perspective extraction:

- Time complexity: $O(W_p \times H_p \times N_{crops})$ per equirectangular image
- For 1024×1024 crops, ~1M pixels per crop
- 6 crops per image: ~6M operations
- Typical: 0.5-2 seconds per crop (CPU)

Pose transfer:

- Time complexity: $O(N_{images})$ (constant time per image)
- Negligible compared to image processing

Total pipeline:

- For 100 equirectangular images, 6 crops each = 600 perspective images
- Extraction: ~300-1200 seconds (5-20 minutes)
- Pose transfer: <1 second
- **Total: ~5-20 minutes for full dataset**

9.2 Optimization Strategies

Parallelization:

```
from multiprocessing import Pool
from functools import partial

def process_single_image(equ_image_path, config, sfm_reader):
"""Process single equirectangular image."""
```

Implementation here
pass

def parallel_processing(equ_images, config, num_workers=8):
"""
Process images in parallel.

```
    Args:
        equ_images: List of image paths
        config: Configuration dictionary
        num_workers: Number of parallel processes
    """
    with Pool(num_workers) as pool:
        process_func = partial(
            process_single_image,
            config=config,
            sfm_reader=sfm_reader
        )
        results = pool.map(process_func, equ_images)

    return results
```

GPU Acceleration (if using custom implementation):

import cupy as cp # GPU-accelerated NumPy

def extract_perspective_gpu(erp_image_gpu, fov, theta, phi, width, height):
"""
GPU-accelerated perspective extraction using CuPy.

```
    Args:
        erp_image_gpu: CuPy array on GPU
        (other parameters same as CPU version)

    Returns:
        CuPy array (perspective image)
    """
    # Same algorithm but with CuPy arrays
    # Vectorized operations run on GPU
```

```
# 10-50x speedup possible
pass
```

---

10. Deployment and Usage

10.1 Installation Requirements

requirements.txt:

```
numpy>=1.21.0
opencv-python>=4.5.0
pycolmap>=0.6.0
Equirec2Perspec>=0.1.0
Pillow>=9.0.0
```

Installation:

```
# Install dependencies
pip install -r requirements.txt

# Verify pycolmap installation
python -c "import pycolmap; print(pycolmap.version)"
```

10.2 Command-Line Interface

main.py:

```
#!/usr/bin/env python3
import argparse
from pipeline_orchestrator import PipelineOrchestrator

def main():
parser = argparse.ArgumentParser(
description='Transfer SphereSfM poses to perspective images'
)
parser.add_argument(
'--config',
required=True,
help='Path to configuration JSON file'
)
parser.add_argument(
'--refine',
action='store_true',
help='Apply bundle adjustment refinement'
)
parser.add_argument(
'--visualize',
```

action='store_true',
help='Launch COLMAP GUI after processing'
)

```
    args = parser.parse_args()

    # Run pipeline
    orchestrator = PipelineOrchestrator(args.config)
    orchestrator.process_all_images()

    # Optional refinement
    if args.refine:
        from perspective_reconstruction import refine_poses_bundle_adjustment
        refine_poses_bundle_adjustment(
            orchestrator.config['output_sparse_path'],
            orchestrator.config['output_sparse_path'] + '_refined'
        )

    # Optional visualization
    if args.visualize:
        import subprocess
        subprocess.run([
            'colmap', 'gui',
            '--import_path', orchestrator.config['output_sparse_path']
        ])
```

if **name** == '**main**':
main()

Usage:

Basic usage

python main.py --config config.json

With bundle adjustment refinement

python main.py --config config.json --refine

With visualization

python main.py --config config.json --visualize

10.3 Configuration File Documentation

Complete config.json structure:

```
{
"spheresfm_sparse_path": "colmap/sparse/0",
"equirectangular_image_dir": "images/equirectangular",
"perspective_output_dir": "images/perspective",
"output_sparse_path": "sparse_perspective/0",
"crop_configurations": [
{
"fov": 90,
"theta": 0,
"phi": 0,
"width": 1024,
"height": 1024,
"description": "Forward view"
},
{
"fov": 90,
"theta": 90,
"phi": 0,
"width": 1024,
"height": 1024,
"description": "Right view"
},
{
"fov": 90,
"theta": 180,
"phi": 0,
"width": 1024,
"height": 1024,
"description": "Back view"
},
{
"fov": 90,
"theta": -90,
"phi": 0,
"width": 1024,
"height": 1024,
"description": "Left view"
},
{
"fov": 90,
"theta": 0,
"phi": 45,
```

"width": 1024,
"height": 1024,
"description": "Up view"
},
{
"fov": 90,
"theta": 0,
"phi": -45,
"width": 1024,
"height": 1024,
"description": "Down view"
}
],
"processing_options": {
"parallel": true,
"num_workers": 8,
"batch_size": 50
}
}

---

11. Troubleshooting Guide

11.1 Common Issues

Issue: pycolmap import error

Solution:

Reinstall with specific version

pip install pycolmap==0.6.1

Or use conda

conda install -c conda-forge pycolmap

Issue: Perspective images appear distorted

Possible causes:

- Incorrect FOV calculation
- Wrong theta/phi sign convention
- Image coordinate system mismatch

Debug:

Verify intrinsics

intrinsics = compute_intrinsics(90, 1024, 1024)
print(f"Focal length: {intrinsics['fx']}") # Should be ~512 for 90° FOV

Test with known values

test_extraction(fov=90, theta=0, phi=0) # Should show forward view

Issue: Poses not matching in COLMAP GUI

Solution:

- Verify quaternion format ([w, x, y, z] vs [x, y, z, w])
- Check coordinate system convention (COLMAP uses camera-to-world)
- Validate rotation matrix determinant = 1

11.2 Debugging Tools

Pose visualization:

def visualize_poses(reconstruction_path):
"""
Print summary of camera poses in reconstruction.

```
Args:
    reconstruction_path: Path to sparse reconstruction folder
"""
recon = pycolmap.Reconstruction(reconstruction_path)

print(f"Reconstruction summary:")
print(f"  - {len(recon.cameras)} cameras")
print(f"  - {len(recon.images)} images")
print(f"  - {len(recon.points3D)} 3D points")

print("\nCamera poses:")
for image_id, image in recon.images.items():
    if image.has_pose:
        cam_from_world = image.cam_from_world()
        center = -cam_from_world.rotation.matrix().T @ cam_from_world.translati
        print(f"  {image.name}: center = {center}")
```

Intrinsics checker:

def check_intrinsics(reconstruction_path):
"""Verify camera intrinsics are reasonable."""
recon = pycolmap.Reconstruction(reconstruction_path)

```
for cam_id, camera in recon.cameras.items():
    print(f"Camera {cam_id}:")
    print(f"  Model: {camera.model_name}")
    print(f"  Size: {camera.width}x{camera.height}")
    print(f"  Params: {camera.params}")

    if camera.model_name == 'PINHOLE':
        fx, fy, cx, cy = camera.params
        fov_x = 2 * np.arctan(camera.width / (2 * fx))
        fov_y = 2 * np.arctan(camera.height / (2 * fy))
        print(f"  Computed FOV: {np.degrees(fov_x):.1f}° × {np.degrees(fov_y):.1f}°
```

---

12. References and Resources

12.1 Key Papers

[1] Jiang, S., You, K., Li, Y., Weng, D., & Chen, W. (2024). 3D reconstruction of spherical images: A review of techniques, applications, and prospects. *Geo-spatial Information Science*, 1-30.

[2] Jiang, S., You, K., Chen, W., Weng, D., & Li, Y. (2024). 3D reconstruction of spherical images based on incremental structure from motion. *International Journal of Remote Sensing*, 45(8), 2596-2621.

[3] Jiang, S., Liu, J., Li, Y., Weng, D., & Chen, W. (2023). Reliable feature matching for spherical images via local geometric rectification and learned descriptor. *Remote Sensing*, 15(20), 4954.

12.2 External Libraries

- **pycolmap**: https://github.com/colmap/pycolmap
- **Equirec2Perspec**: https://github.com/fuenwang/Equirec2Perspec
- **SphereSfM**: https://github.com/json87/SphereSfM
- **COLMAP Documentation**: https://colmap.github.io/

12.3 Related Resources

- Paul Bourke's sphere-to-perspective converter: https://paulbourke.net/panorama/sphere2persp/
- OmniCV library: https://github.com/kaustubh-sadekar/OmniCV
- COLMAP database format: https://colmap.github.io/database.html
- Perspective-and-Equirectangular tools: https://github.com/timy90022/Perspective-and-Equirectangular

Appendix A: Complete Example Workflow

Step-by-Step Execution

Step 1: Setup directory structure

```
mkdir -p
project/{images/equirectangular,images/perspective,colmap/sparse/0,sparse_perspective}
```

Step 2: Run SphereSfM (prerequisite)

Assuming SphereSfM already completed

Output: colmap/sparse/0/ with cameras.bin, images.bin, points3D.bin

Step 3: Create configuration file

```
cat > config.json << EOF
{
"spheresfm_sparse_path": "colmap/sparse/0",
"equirectangular_image_dir": "images/equirectangular",
"perspective_output_dir": "images/perspective",
"output_sparse_path": "sparse_perspective/0",
"crop_configurations": [
{"fov": 90, "theta": 0, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": 90, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": 180, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": -90, "phi": 0, "width": 1024, "height": 1024},
{"fov": 90, "theta": 0, "phi": 45, "width": 1024, "height": 1024},
{"fov": 90, "theta": 0, "phi": -45, "width": 1024, "height": 1024}
]
}
EOF
```

Step 4: Run pose transfer pipeline

```
python main.py --config config.json
```

Expected output:

```
2026-02-03 19:45:00 - INFO - Found 10 equirectangular images
2026-02-03 19:45:01 - INFO - Processing IMG_0001.jpg...
2026-02-03 19:45:03 - INFO - -> Extracted 6 perspective images
2026-02-03 19:45:04 - INFO - Processing IMG_0002.jpg...
...
2026-02-03 19:50:00 - INFO - Processed 10/10 images
2026-02-03 19:50:01 - INFO - Reconstruction written to sparse_perspective/0
2026-02-03 19:50:01 - INFO - - 1 cameras
```

```
2026-02-03 19:50:01 - INFO - - 60 images
2026-02-03 19:50:01 - INFO - Pipeline complete.
```

Step 5: Verify in COLMAP GUI

```
colmap gui --import_path sparse_perspective/0 --image_path images/perspective
```

Step 6: (Optional) Run dense reconstruction

Undistort images (perspective already undistorted)

```
colmap image_undistorter
--image_path images/perspective
--input_path sparse_perspective/0
--output_path dense/0
```

Stereo matching

```
colmap patch_match_stereo --workspace_path dense/0
```

Fusion

```
colmap stereo_fusion
--workspace_path dense/0
--output_path dense/0/fused.ply
```

Appendix B: Performance Benchmarks

Test configuration:

- CPU: Intel Core Ultra 7 265K
- GPU: RTX 5070 Ti 16GB
- RAM: 32GB DDR5 6000MHz
- Dataset: 100 equirectangular images (7296×3648)
- Crops: 6 per image (1024×1024)

Results:

Operation	Time (CPU)	Time (GPU)
SphereSfM (prerequisite)	45 min	N/A
Perspective extraction	18 min	3 min
Pose transfer	2 sec	2 sec
Database creation	5 sec	5 sec
Bundle adjustment (optional)	8 min	N/A
Total (without BA)	**18.1 min**	**3.1 min**
Total (with BA)	**26.1 min**	**11.1 min**

Table 2: Pipeline performance benchmarks

Speedup vs. full SfM on perspective images: 3-5x faster

Appendix C: Coordinate System Conventions

COLMAP Coordinate System

Camera coordinate system:

- X-axis: right
- Y-axis: down
- Z-axis: forward (camera looking direction)

World coordinate system:

- Arbitrary origin (typically first camera)
- Right-handed coordinate system

Transformation:

- `cam_from_world`: World → Camera transformation
- Quaternion: [w, x, y, z] format (scalar-first)

Equirectangular Conventions

Angle definitions:

- Theta (θ): Pan/yaw angle, rotation around vertical (Z) axis
  - 0° = forward
  - +90° = right
  - -90° = left
  - 180° = backward
- Phi (φ): Tilt/pitch angle, rotation around horizontal (X) axis
  - 0° = horizontal
  - +45° = up
  - -45° = down

Image coordinates:

- Origin: top-left corner
- U-axis: horizontal (left-to-right)
- V-axis: vertical (top-to-bottom)

Document Revision History

Version	Date	Changes
1.0	2026-02-03	Initial specification document

End of Specification Document